Algoritmo Voraz de Codificación de Huffman

Daniel Arturo Barrera Berrio
Sharon Nicolle Martínez Mesa
Henry Gustavo García Fuentes

19 de octubre de 2026



ÍNDICE

Contexto histórico

¿Qué es la codificación de Huffman?

¿Cómo se ve el algoritmo?

Bibliografía

Agradecimientos



CONTEXTO HISTÓRICO

David Huffman lo inventó en 1951 siendo estudiante de doctorado en el MIT, para librarse de un examen final, y demostró que ningún otro código de ese tipo puede ser más corto.

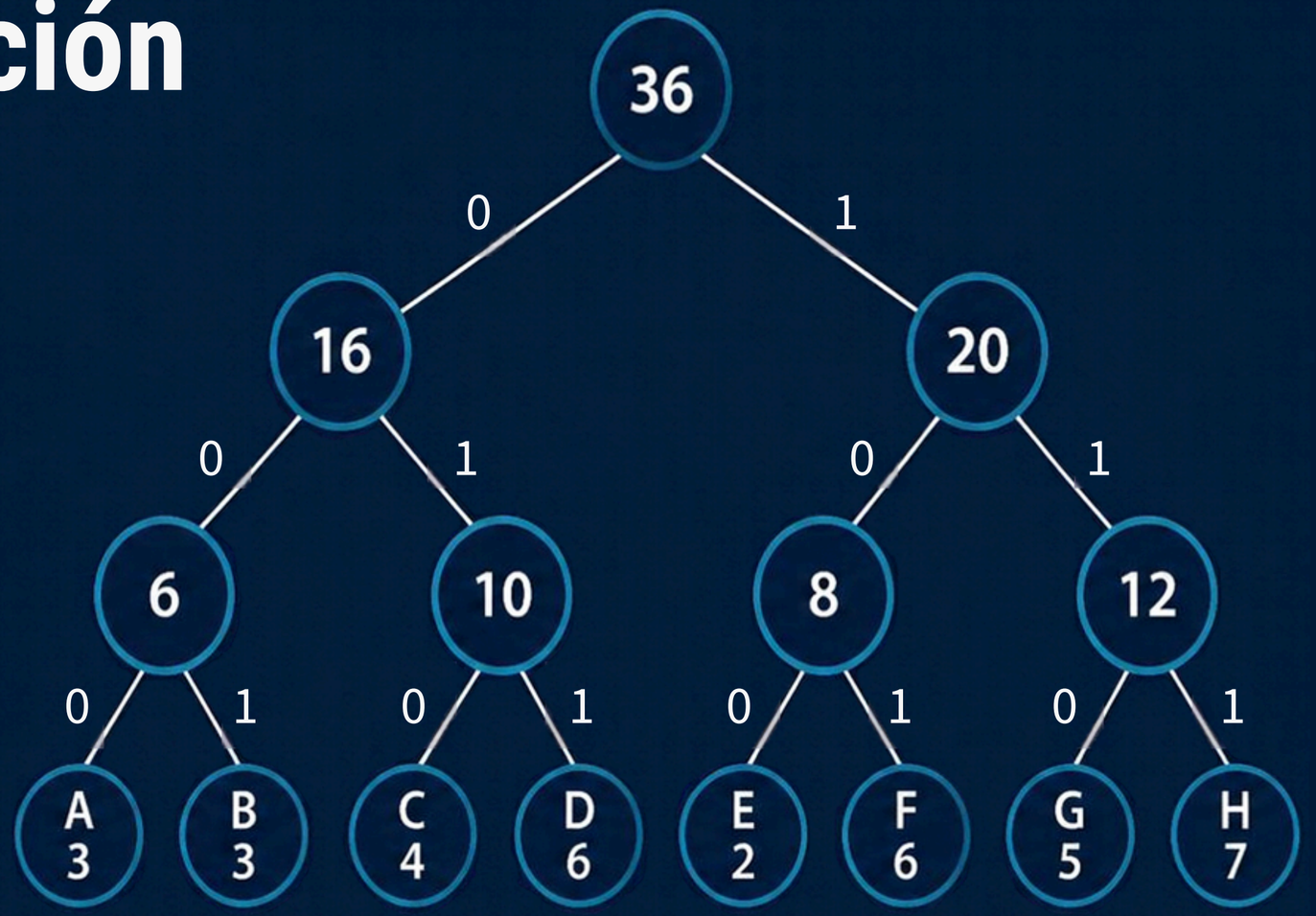


¿Qué es la codificación de Huffman?

Es un algoritmo de compresión sin pérdidas de datos . Asigna códigos de longitud variable a los caracteres de entrada en función de su frecuencia de aparición . La idea básica es:

- A los caracteres que aparecen con mayor frecuencia se les asignan códigos más cortos.
- A los caracteres que aparecen con menos frecuencia se les asignan códigos más largos.

Esto reduce la longitud media de los datos codificados y, por lo tanto, logra la compresión.



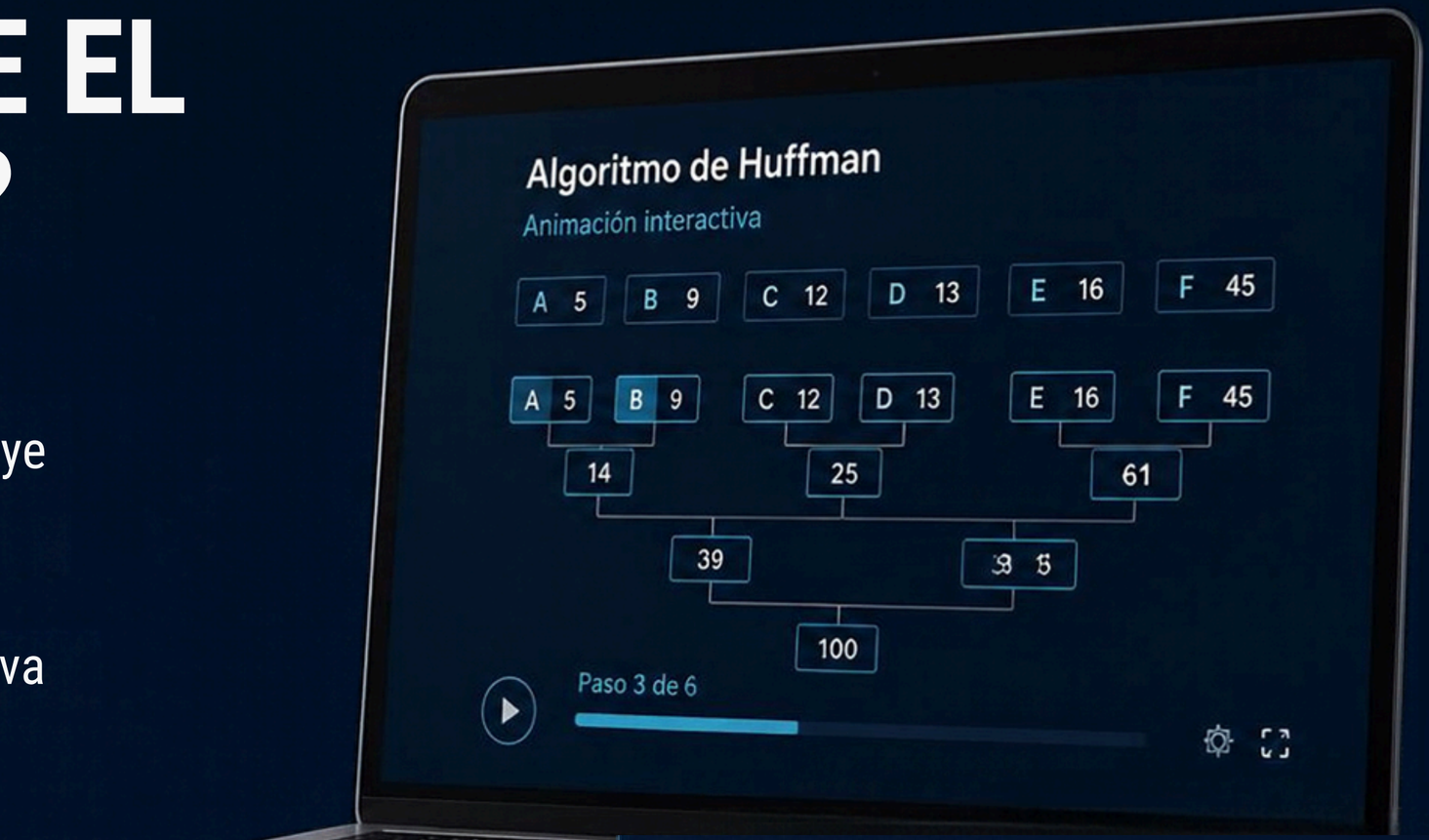
Símbolo	A	B	C	D	E	F	G	H
Código	000	001	010	011	100	101	110	111

¿CÓMO SE VE EL ALGORITMO?

Explora cómo el algoritmo construye el árbol óptimo en cada paso.

También se refleja el código en Java del algoritmo.

Escanea el código QR o ingresa al enlace para acceder.



Accede a la animación aquí:

https://henrygarciaf.github.io/Simulador-Huffman/

ALGORITMO

```java
public class Huffman {

    // Representa un nodo del árbol de Huffman
    static class Nodo {
        char caracter;
        int frecuencia;
        Nodo izquierda, derecha;

        // Nodo hoja: representa un carácter
        Nodo(char caracter, int frecuencia) {
            this.caracter = caracter;
            this.frecuencia = frecuencia;
        }

        // Nodo interno: combina dos nodos
        Nodo(Nodo izquierda, Nodo derecha) {
            this.caracter = '\0';
            this.frecuencia = izquierda.frecuencia + derecha.frecuencia;
            this.izquierda = izquierda;
            this.derecha = derecha;
        }

        // Si no tiene hijos, es una hoja
        boolean esHoja() {
            return izquierda == null && derecha == null;
        }
    }
```

```java
    static Nodo construirArbol(String texto) {
        if (texto == null || texto.isEmpty()) {
            return null;
        }

        // Contar la frecuencia de cada carácter
        Map<Character, Integer> frecuencias = new HashMap<>();

        for (char c : texto.toCharArray()) {
            frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);
        }

        // Heap mínimo: el nodo de menor frecuencia tiene prioridad
        PriorityQueue<Nodo> heap =
            new PriorityQueue<>(Comparator.comparingInt(n -> n.frecuencia));

        // Insertar un nodo por cada carácter distinto
        for (Map.Entry<Character, Integer> entrada : frecuencias.entrySet()) {
            heap.offer(new Nodo(entrada.getKey(), entrada.getValue()));
        }

        // Combinar los dos nodos de menor frecuencia
        while (heap.size() > 1) {
            Nodo a = heap.poll();
            Nodo b = heap.poll();

            Nodo padre = new Nodo(a, b);
            heap.offer(padre);
        }

        // El último nodo es la raíz del árbol
        return heap.poll();
```

ALGORITMO

```java
// Recorrer el árbol para obtener los códigos binarios
static void generarCodigos(
    Nodo nodo, String codigo, Map<Character, String> codigos) {

    if (nodo == null) {
        return;
    }

    if (nodo.esHoja()) {
        // Caso especial: texto con un solo carácter distinto
        codigos.put(nodo.caracter,
            codigo.isEmpty() ? "0" : codigo);
        return;
    }

    generarCodigos(nodo.izquierda, codigo + "0", codigos);
    generarCodigos(nodo.derecha, codigo + "1", codigos);
}
```

```java
    public static void main(String[] args) {
        String texto = "AAAAAB";

        Nodo raiz = construirArbol(texto);
        Map<Character, String> codigos = new HashMap<>();

        generarCodigos(raiz, "", codigos);

        System.out.println("Texto: " + texto);
        System.out.println("Códigos de Huffman:");

        for (Map.Entry<Character, String> entrada : codigos.entrySet()) {
            System.out.println(
                entrada.getKey() + ": " + entrada.getValue());
        }
    }
}
```

COMPLEJIDAD BIG O

Operación	Complejidad
Contar los caracteres	$O(n)$
Insertar o extraer del heap	$O(\log k)$ por operación
Construir el árbol	$O(k \log k)$
Tiempo total	$O(n + k \log k)$
Espacio auxiliar	$O(k)$

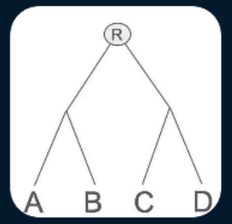
Comportamiento en casos límite

¿Qué ocurre con diferentes tipos de entradas?



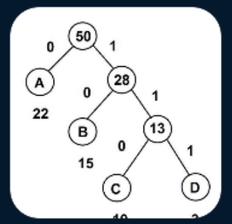
Caso 1: Un solo carácter (`AAAA`)

No hay combinaciones; se asigna el código `0`.



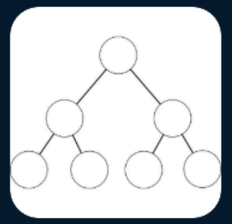
Caso 2: Frecuencias iguales (`ABCD`)

Los empates pueden generar diferentes árboles válidos.



Caso 3: Frecuencias desiguales

Los caracteres frecuentes suelen tener códigos más cortos.



Caso 4: Texto vacío

No se construye ningún árbol.

BIBLIOGRAFÍA

Ledesma, J. S. (2026, 5 septiembre). La codificación de Huffman. Ikusmira. https://ikusmira.org/p/la-codificacion-de-huffman/

Huffman Coding Algorithm - Algorithm Room. (s. f.). https://algorithmroom.com/dsa/huffman-coding-algorithm



Gracias por su atención

Esperamos que esta presentación haya sido de utilidad.

Quedamos atentos a sus preguntas.

Contacto: equipo@huffman.com

